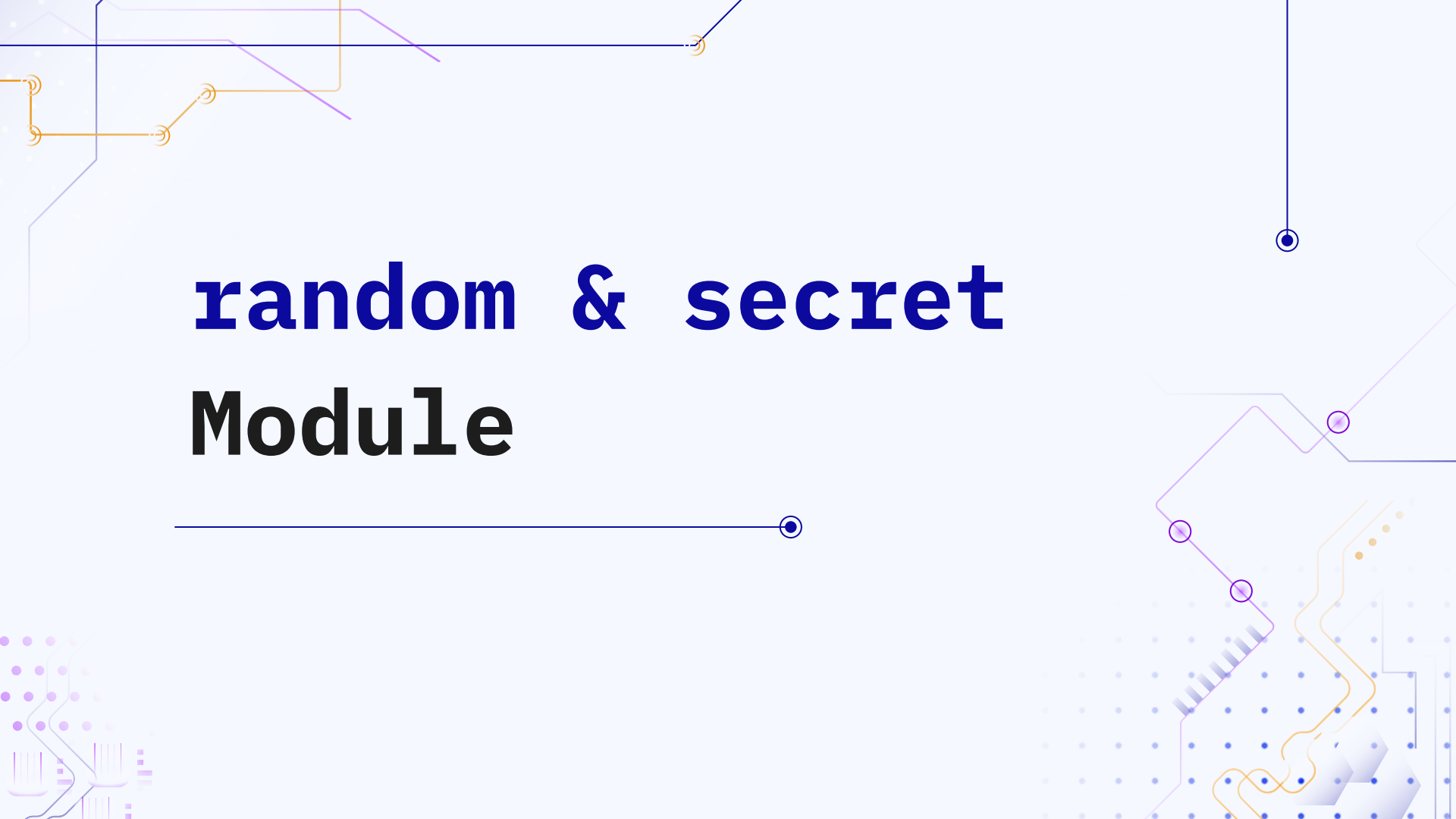




random & secret **Module**

Introduction

Python's **random** module generates pseudo-random numbers (looks random, but actually produced by an algorithm). It's perfect for games, simulations, shuffling, sampling, etc.

Syntax

```
import random
```

Core functions (Most Used)

1. Random float (random)

```
import random  
  
print(random.random()) # float in [0.0, 1.0)
```

2. Random integers (randint, randrange)

```
print(random.randint(5, 10)) # int in [5, 10]  
print(random.randrange(10)) # int in [0, 10)
```

3. Random float in a range (uniform)

```
print(random.uniform(5, 10))
```

Core functions (Most Used)

4. Pick from a sequence (choice, choices, sample)

```
print(random.choice([1, 2, 3]))      # one random item
print(random.choices([1, 2, 3], k=2)) # multiple picks (can repeat),
                                     # supports weights=...
print(random.sample([1, 2, 3], k=2))  # unique picks (no repeat)
```

5. Shuffle

```
import random
lst = [1, 2, 3]
random.shuffle(lst)
print(lst)
```

NOTE: `random.shuffle()` **shuffles a list in place** and returns **None**, so `print(random.shuffle(...))` will always print None.

Reproducibility

syntax

```
random.seed(42)
print(random.random())    # always the same with seed 42
print(random.randint(1, 10))
```

random.seed(x) fixes the randomness so you get the **same results every run** — useful in testing.

Security

Random module is not ideal for security (passwords/OTP) because random can be predictable if someone knows the seed/state.

Use the secrets module for **cryptographically secure** randomness.

python

```
import secrets

# Secure random int in [0, n)
n = 10
x = secrets.randbelow(n)
print(x)

# Secure random choice from a sequence
print(secrets.choice([1, 2, 3]))
```

Security

python

```
# Generate tokens / keys:  
import secrets  
  
print(secrets.token_hex(16))      # 32 hex chars  
print(secrets.token_urlsafe(16))  # URL-safe string
```

Rule of thumb:

- Use random for games/simulations/shuffling.
- Use secrets for anything involving **authentication, tokens, passwords, security**.



Practice Set – 4

Download Link in **Description** and **Pinned Comment**

WATCH

Level up your coding with each episode in this focused Python series.



Next Video!

**Practice Set - 4
Solution**

